

Title: Robust Selenium Automation Framework and Test Plan for Enterprise Dashboard Application

1. Introduction

You are building a large-scale enterprise application with the following stack:

Frontend: Next.js (TypeScript, App Router)

State Management: Redux Toolkit

Backend: Java Spring Boot

Database: Snowflake

Authentication: SSO (Single Sign-On) + Role-Based Access Control (RBAC)

CI/CD: GitHub Actions + AWS

UI Testing Tools: Jest (unit testing)

API Testing Tools: JUnit (unit testing), REST Assured (integration testing)

Now you want to implement Selenium-based functional UI automation using Java. Below is a detailed, beginner-friendly plan.

2. Automation Goals

Automate common UI workflows (dashboard navigation, filters, CRUD operations)

Validate SSO login and role-based access restrictions

Automate workflow wizards

Run tests in CI/CD pipeline (GitHub Actions)

3. Tools and Technologies

Selenium WebDriver (for browser automation)

TestNG (test runner framework)

Maven (build tool + dependency manager)

WebDriverManager (manages browser drivers)

Allure Reports (optional, for beautiful test reports)

4. Project Structure

Create a Maven Java project with the following structure:

```
selenium-framework/  
├── src/  
│   ├── main/java/  
│   │   ├── base/           # Base test class for setup/teardown  
│   │   ├── pages/         # Page Object Model (POM) classes  
│   │   ├── utils/         # Utility classes (Config, JWT, waits)  
│   │   └── factory/        # WebDriver factory for parallel tests  
│   └── test/java/  
│       └── tests/          # Actual test cases  
├── resources/  
│   └── config.properties  # URLs, browser, credentials  
├── pom.xml                 # Maven build file  
└── testng.xml              # TestNG suite file
```

5. Step-by-Step Implementation

Step 1: Setup Maven Project

```

<!-- pom.xml dependencies -->
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.19.1</version>
  </dependency>
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.7.0</version>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.8.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

Step 2: Create WebDriver Factory

```

// DriverFactory.java
public class DriverFactory {
    private static WebDriver driver;

    public static WebDriver initDriver() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
        return driver;
    }

    public static void quitDriver() {
        driver.quit();
    }

    public static WebDriver getDriver() {
        return driver;
    }
}

```

Step 3: Base Test Class

```
// BaseTest.java
public class BaseTest {
    protected WebDriver driver;

    @BeforeMethod
    public void setup() {
        driver = DriverFactory.initDriver();
        driver.get("https://your-application-url.com");
    }

    @AfterMethod
    public void tearDown() {
        DriverFactory.quitDriver();
    }
}
```

Step 4: Page Object Model (POM)

```
// LoginPage.java
public class LoginPage {
    private WebDriver driver;

    @FindBy(id = "username")
    private WebElement usernameField;

    @FindBy(id = "password")
    private WebElement passwordField;

    @FindBy(id = "login-button")
    private WebElement loginBtn;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    public void login(String username, String password) {
        usernameField.sendKeys(username);
        passwordField.sendKeys(password);
        loginBtn.click();
    }
}
```

Repeat this for DashboardPage, WorkflowPage, etc.

Step 5: Sample Test Case

```
// LoginTest.java
public class LoginTest extends BaseTest {
    @Test
    public void testSSOLogin() {
        LoginPage loginPage = new LoginPage(driver);
        loginPage.login("testuser", "testpass");
        Assert.assertTrue(driver.getCurrentUrl().contains("dashboard"));
    }
}
```

6. Test Scenarios (Beginner Friendly)

Login & SSO

Login via SSO

Redirect to dashboard

Validate role-based menu visibility

Dashboard Page

Verify charts load

Interact with filters

Validate table sorting/pagination

Workflow Wizard

Navigate between wizard steps

Validate required fields

Save, submit, approve workflow

7. CI/CD Integration (GitHub Actions)

```
# .github/workflows/ui-tests.yml
name: UI Tests
```

```
on: [push]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Set up Java
```

```
        uses: actions/setup-java@v3
```

```
        with:
```

```
          java-version: '17'
```

```
      - name: Run Tests
```

```
        run: mvn test
```

8. Team Plan

Team Role	Responsibility
-----------	----------------

QA Architect	Set up framework
--------------	------------------

UI Test Engineers	Write page classes & tests
-------------------	----------------------------

Backend QA	Validate APIs + DB states
------------	---------------------------

DevOps	CI/CD pipeline integration
--------	----------------------------

9. Next Steps

Setup the framework locally with basic tests

Add pages incrementally: login, dashboard, wizard

Configure CI pipeline

Expand coverage and enable parallel test execution

Let me know if you'd like the full working starter code or help writing tests for your dashboard UI components (tables, charts, filters, etc.).

Here's a fully detailed, beginner-friendly plan for implementing a robust Selenium test automation framework using Java, tailored to your project's stack and requirements. You can review the full plan in the "Selenium Test Plan" document now. Let me know if you'd like help writing sample tests for your dashboards or need a working starter project.